Makine Öğrenmesi Ödev!

Soru => (e-e) mi (e-v) mi gelmesi daha olası

Gözlem dizisi de [High, Low] → yani t1 = High t2 = Low.

Başlangıç olasılığıda P(e) = 1.0.

Çözüm:

	High	Low
e	0.7	0.3
v	0.1	0.9

} Gözlem aşaması

	e	v
e	0.6	0.4
v	0.8	0.2

} Geçiş aşaması

P(High|e) = 0.7
P(High|v) = 0.1
P(low|e) = 0.3
P(low|v) = 0.9

P(e|e) = 0.6
P(e|v) = 0.4
P(v|e) = 0.8
P(v|v) = 0.2.

t1 adımında (Başlangıç durumu ayrıca).

P(High|e) ya da P(~~High~~|v) olabilir. → 0

Başlangıç olasılığı P(e) = 1.0
yani v = 0 → P(v) = 0

↳ $\underbrace{P(e)}_{1.0} \cdot \underbrace{P(High|e)}_{0.7}$ => 1.0 x 0.7 = 0.7
↳ yani t1 sonunda e = 0.7

t2 adımında.

e gelebilir ya da v gelebilir
↳ P(low|e) ya da P(low|v)

e gelme durumunda.

e → e = P(e|e) → 0.6.

↳ sonuç = önceki olasılık x geçiş x gözlem.

yani

0.7 x 0.6 x 0.3 = 0.126

v gelme durumunda

e → v = P(e|v) = 0.4

↳ 0.7 x 0.4 x 0.9 = 0.252

	e	v
High(t1)	0.7	0
Low(t2)	0.126	0.252

Sonuç! 0.252 > 0.126'dan t2'de v gelme olasılığı daha yüksek. yani ev gelme olasılığı ee gelme olasılığından daha yüksek.

Analiz ve yorumlama:

1-> HMM'de emisyon olasılığı, bir gizli durumun (burada fonem) belirli bir gözlemi üretme olasılığı.

Gürültü ile:
=> Doğru gözlemin olasılığı azalır. (yanlış gözlem olasılığı 0.3'den 0.4'ü çıkar mesela)

=> Belirsizlik artar. (emisyon matrisindeki değerler daha eşit hale gelir.)

2-> Viterbi algoritması => HMM'de en olası durum dizisini bulmak için kullanılır.

Ve HMM'de durum sayısı arttıkça: maliyet artar.

↳ ayrıca özelliklerl sınırlı kullanır.

↳ uzun bağlam bilgisi zor.

Deep learning Avantajı:
=> Çok sayıda kelimeyi ve özellikleri öğrenebilir.
=> Uzun bağlamı modelleyebilir.
=> Gürültüye karşı daha dayanıklı
=> Büyük veri ile performans HMM'den çok daha iyi olur.